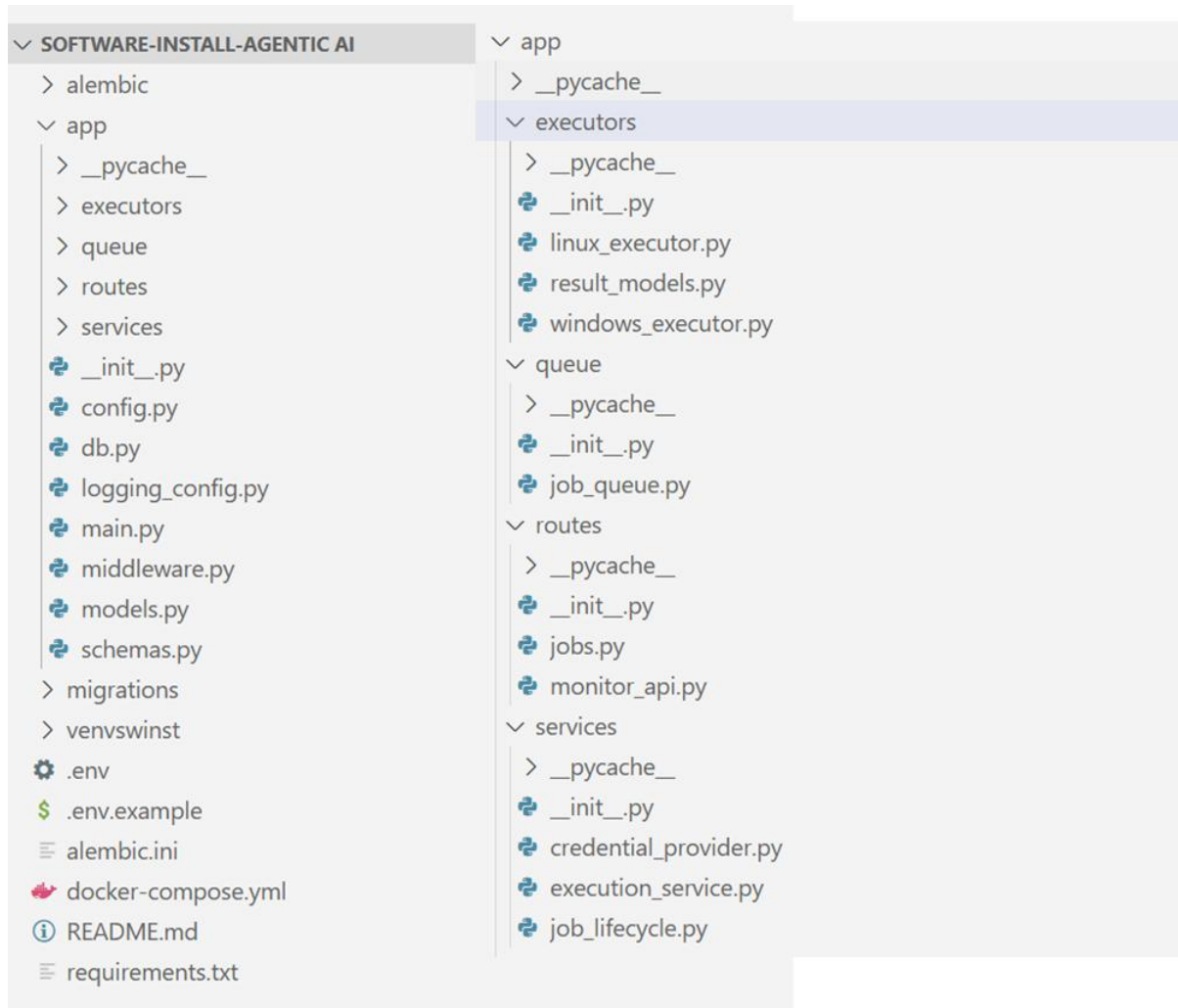


Phase1_Code file structure : C:\Users\458027\OneDrive - Cognizant\Drive\Learn\AIOps_HCS\Phase 2\software-install-Agentic AI



Executors(_init_.py)

- linux_executors.py
- result_models.py
- windows_executors.py

Queue(_init_.py)

- job_queue.py

Routes(_init_.py)

- Jobs.py
- Monitory_api.py

Services(_init_.py)

- Credential_provider.py
- Execution_service.py
- Job_lifecycle.py

1.Alembic:

1.1.Env:

```

from logging.config import fileConfig

from alembic import context
from sqlalchemy import engine_from_config, pool

from app.config import settings
from app.db import Base
from app import models # noqa: F401 - ensure models are imported
from app.models import Base

config = context.config

db_url = settings.get_database_url() if callable(settings.get_database_url)
else settings.get_database_url
config.set_main_option("sqlalchemy.url", db_url)

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata

def run_migrations_offline():
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()

def run_migrations_online():
    connectable = engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:

```

```

        context.configure(connection=connection,
                           target_metadata=target_metadata)

        with context.begin_transaction():
            context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()

```

1.2. versions\0001_create_jobs_and_job_steps.py

```

"""create jobs and job_steps tables

```

```

Revision ID: 0001_create_jobs_and_job_steps
Revises:
Create Date: 2026-05-31
"""

```

```

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

```

```

revision = "0001_create_jobs_and_job_steps"
down_revision = None
branch_labels = None
depends_on = None

```

```

#job_status = sa.Enum("PENDING", "RUNNING", "SUCCESS", "FAILED",
name="job_status")

```

```

job_status = postgresql.ENUM(
    "PENDING",
    "RUNNING",
    "SUCCESS",
    "FAILED",
    name="job_status",
    create_type=False
)

```

```

def upgrade():
    job_status.create(op.get_bind(), checkfirst=True)

    op.create_table(

```

```

    "jobs",
    sa.Column("id", sa.String(), nullable=False),
    sa.Column("ticket_id", sa.String(), nullable=False),
    sa.Column("module", sa.String(), nullable=False),
    sa.Column("status", job_status, nullable=False),
    sa.Column("target_host", sa.String(), nullable=False),
    sa.Column("os_type", sa.String(), nullable=False),
    sa.Column("software_name", sa.String(), nullable=False),
    sa.Column("software_version", sa.String(), nullable=True),
    sa.Column("requested_by", sa.String(), nullable=True),
    sa.Column("justification", sa.Text(), nullable=True),
    sa.Column("trace_id", sa.String(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False),
    sa.Column("updated_at", sa.DateTime(), nullable=False),
    sa.PrimaryKeyConstraint("id"),
)
op.create_index(op.f("ix_jobs_ticket_id"), "jobs", ["ticket_id"],
unique=False)

op.create_table(
    "job_steps",
    sa.Column("id", sa.Integer(), autoincrement=True, nullable=False),
    sa.Column("job_id", sa.String(), nullable=False),
    sa.Column("step_name", sa.String(), nullable=False),
    sa.Column("status", sa.String(), nullable=False),
    sa.Column("message", sa.Text(), nullable=True),
    sa.Column("exit_code", sa.Integer(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False),
    sa.ForeignKeyConstraint(["job_id"], ["jobs.id"], ondelete="CASCADE"),
    sa.PrimaryKeyConstraint("id"),
)
op.create_index(op.f("ix_job_steps_job_id"), "job_steps", ["job_id"],
unique=False)

def downgrade():
    op.drop_index(op.f("ix_job_steps_job_id"), table_name="job_steps")
    op.drop_table("job_steps")
    op.drop_index(op.f("ix_jobs_ticket_id"), table_name="jobs")
    op.drop_table("jobs")
    job_status.drop(op.get_bind(), checkfirst=True)

```

1.3. versions\0002_extend_job_status_lifecycle.py

```

"""extend job status lifecycle values

Revision ID: 0002_extend_job_status_lifecycle
Revises: 0001_create_jobs_and_job_steps
Create Date: 2026-05-31
"""

from alembic import op
import sqlalchemy as sa

revision = "0002_extend_job_status_lifecycle"
down_revision = "0001_create_jobs_and_job_steps"
branch_labels = None
depends_on = None

old_job_status = sa.Enum("PENDING", "RUNNING", "SUCCESS", "FAILED",
name="job_status")
new_job_status = sa.Enum("PENDING", "VALIDATING", "RUNNING", "VERIFYING",
"SUCCESS", "FAILED", name="job_status")

def upgrade():
    bind = op.get_bind()

    # PostgreSQL enum extension
    op.execute("ALTER TYPE job_status ADD VALUE IF NOT EXISTS 'VALIDATING'")
    op.execute("ALTER TYPE job_status ADD VALUE IF NOT EXISTS 'VERIFYING'")

def downgrade():
    # Enum value removal is not trivial in PostgreSQL.
    # Safe downgrade typically requires creating a new enum type and casting
    data.
    # For MVP/Local environment, leave as no-op or handle manually if needed.
    pass

```

1.4. versions\003_add_execution_fields.py

```

"""add execution fields

Revision ID: 87004cf3c2f2
Revises: 0002_extend_job_status_lifecycle
Create Date: 2026-06-05 16:10:25.245710

"""

```

```

from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision: str = '003_add_execution_fields'
down_revision: Union[str, None] = '0002_extend_job_status_lifecycle'
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade():
    op.add_column(
        "jobs",
        sa.Column("retry_count", sa.Integer(), nullable=False,
server_default="0"),
    )
    op.add_column(
        "jobs",
        sa.Column("max_retries", sa.Integer(), nullable=False,
server_default="2"),
    )
    op.add_column(
        "jobs",
        sa.Column("timeout_seconds", sa.Integer(), nullable=False,
server_default="300"),
    )
    op.add_column(
        "jobs",
        sa.Column("last_error", sa.Text(), nullable=True),
    )

def downgrade() -> None:
    ### commands auto generated by Alembic - please adjust! ###
    op.drop_column('jobs', 'last_error')
    op.drop_column('jobs', 'timeout_seconds')
    op.drop_column('jobs', 'max_retries')
    op.drop_column('jobs', 'retry_count')
    ### end Alembic commands ###

```

1.5. versions\004_add_workflow_ready_fields.py

```

"""add execution_mode and new fields

```

```

Revision ID: 2fb2ed7418c2
Revises: 003_add_execution_fields
Create Date: 2026-06-15 18:28:46.381711

```

```

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision: str = '004_add_workflow_ready_fields'
down_revision: Union[str, None] = '003_add_execution_fields'
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.add_column('jobs', sa.Column('execution_mode', sa.String(),
    nullable=True))
    op.add_column('jobs', sa.Column('scheduled_time', sa.DateTime(),
    nullable=True))
    # ### end Alembic commands ###

def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_column('jobs', 'scheduled_time')
    op.drop_column('jobs', 'execution_mode')
    # ### end Alembic commands ###

```

2. Config.py

```

from pydantic_settings import BaseSettings, SettingsConfigDict
class Settings(BaseSettings):
    APP_NAME: str = "software-install-mvp"
    APP_ENV: str = "local"
    APP_HOST: str = "0.0.0.0"
    APP_PORT: int = 8000
    LOG_LEVEL: str = "INFO"
    POSTGRES_HOST: str = "localhost"
    POSTGRES_PORT: int = 5432
    POSTGRES_DB: str = "software_install"
    POSTGRES_USER: str = "postgres"
    POSTGRES_PASSWORD: str = "postgres"

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

```

```

@property
def get_database_url(self) -> str:
    return (
        f"postgresql+psycopg2://{self.POSTGRES_USER}:{self.POSTGRES_PASSWO
RD}"
        f"@{self.POSTGRES_HOST}:{self.POSTGRES_PORT}/{self.POSTGRES_DB}"
    )

settings = Settings()
#print(settings.get_database_url) # Access to trigger any potential
validation issues early

```

3. db.py

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from app.config import settings

engine = create_engine(
    settings.get_database_url,
    pool_pre_ping=True,
    future=True
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
    future=True
)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
#print("DB URL:", settings.get_database_url)

```

4. login_config.py

```
import logging

from app.config import settings

def configure_logging():
    logging.basicConfig(
        level=getattr(logging, settings.LOG_LEVEL.upper(), logging.INFO),
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
    )
```

5. Main.py

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from sqlalchemy import text
from contextlib import asynccontextmanager

from app.config import settings
from app.db import engine
from app.middleware import RequestContextMiddleware
from app.logging_config import configure_logging
from app.queue.job_queue import start_workers, stop_workers
from app.services.execution_service import process_job

from app.routes.jobs import router as jobs_router
from app.routes.monitor_api import router1 as monitoring_router

configure_logging()

#  Lifespan (startup + shutdown)
@asynccontextmanager
async def lifespan(app: FastAPI):
    print(" Starting workers...")
    start_workers(worker_count=3, worker_handler=process_job)

    yield

    print(" Stopping workers...")
    stop_workers()

#  ONLY ONE app instance
app = FastAPI(
    title=settings.APP_NAME,
    version="0.1.0",
```

```

        description="Minimal Agentic AI Software Install MVP - Phase 2",
        lifespan=lifespan
    )

#  Middleware
app.add_middleware(RequestContextMiddleware)

#  Routers (IMPORTANT)
app.include_router(jobs_router)
app.include_router(monitoring_router)

#  Root
@app.get("/", tags=["root"])
def root():
    return {
        "message": "Software Install MVP API is running",
        "docs": "/docs"
    }

#  Health
@app.get("/api/v1/health", tags=["health"])
def health():
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
            db_status = "up"
    except Exception:
        db_status = "down"

    return {
        "status": "ok" if db_status == "up" else "degraded"
    }

#  Global Exception Handler
@app.exception_handler(Exception)
async def generic_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={
            "detail": "Internal server error",
            "trace_id": getattr(request.state, "trace_id", None)
        }
    )

```

6.middleware.py

```
import logging
import time
import uuid
from starlette.middleware.base import BaseHTTPMiddleware

logger = logging.getLogger("app.middleware")

class RequestContextMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        trace_id = request.headers.get("x-trace-id", str(uuid.uuid4()))
        request.state.trace_id = trace_id

        start = time.time()
        response = await call_next(request)
        duration_ms = round((time.time() - start) * 1000, 2)
        logger.info(
            "method=%s path=%s trace_id=%s duration_ms=%s status_code=%s",
            request.method,
            request.url.path,
            trace_id,
            duration_ms,
            response.status_code,
        )

        response.headers["x-trace-id"] = trace_id
        response.headers["x-response-time-ms"] = str(duration_ms)

    return response
```

7. models.py

```
import enum
import uuid
from datetime import datetime
from sqlalchemy import Column, String, DateTime, Text, ForeignKey, Integer, Enum
from sqlalchemy.orm import relationship

from app.db import Base

class JobStatus(str, enum.Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCESS = "SUCCESS"
```

```

FAILED = "FAILED"
VALIDATING = "VALIDATING"
VERIFYING = "VERIFYING"

```

```

class Job(Base):
    __tablename__ = "jobs"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    ticket_id = Column(String, nullable=False, index=True)
    module = Column(String, nullable=False, default="sw-install")
    status = Column(Enum(JobStatus, name="job_status"), nullable=False,
default=JobStatus.PENDING)

    target_host = Column(String, nullable=False)
    os_type = Column(String, nullable=False)
    software_name = Column(String, nullable=False)
    software_version = Column(String, nullable=True)

    requested_by = Column(String, nullable=True)
    justification = Column(Text, nullable=True)
    trace_id = Column(String, nullable=True)

    #phase2B
    retry_count = Column(Integer, nullable=False, default=0)
    max_retries = Column(Integer, nullable=False, default=3)
    timeout_seconds = Column(Integer, nullable=False, default=300)
    last_error = Column(Text, nullable=True)

    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    updated_at = Column(
        DateTime,
        nullable=False,
        default=datetime.utcnow,
        onupdate=datetime.utcnow
    )
    execution_mode = Column(String, default="immediate") # immediate /
scheduled
    scheduled_time = Column(DateTime, nullable=True)

    steps = relationship("JobStep", back_populates="job", cascade="all,
delete-orphan")

```

```

class JobStep(Base):
    __tablename__ = "job_steps"

```

```

id = Column(Integer, primary_key=True, autoincrement=True)
job_id = Column(String, ForeignKey("jobs.id", ondelete="CASCADE"),
nullable=False, index=True)

step_name = Column(String, nullable=False)
status = Column(String, nullable=False, default="PENDING")
message = Column(Text, nullable=True)
exit_code = Column(Integer, nullable=True)

created_at = Column(DateTime, nullable=False, default=datetime.utcnow)

job = relationship("Job", back_populates="steps")

class JobStatus(str, enum.Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCESS = "SUCCESS"
    FAILED = "FAILED"
    VALIDATING = "VALIDATING"
    VERIFYING = "VERIFYING"
    FAILED_FINAL = "FAILED_FINAL"

```

8.schemas.py

```

import enum
import uuid
from datetime import datetime
from sqlalchemy import Column, String, DateTime, Text, ForeignKey,
Integer, Enum
from sqlalchemy.orm import relationship

from app.db import Base

class JobStatus(str, enum.Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCESS = "SUCCESS"
    FAILED = "FAILED"
    VALIDATING = "VALIDATING"
    VERIFYING = "VERIFYING"

class Job(Base):
    __tablename__ = "jobs"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))

```

```

ticket_id = Column(String, nullable=False, index=True)
module = Column(String, nullable=False, default="sw-install")
status = Column(Enum(JobStatus, name="job_status"), nullable=False,
default=JobStatus.PENDING)

target_host = Column(String, nullable=False)
os_type = Column(String, nullable=False)
software_name = Column(String, nullable=False)
software_version = Column(String, nullable=True)

requested_by = Column(String, nullable=True)
justification = Column(Text, nullable=True)
trace_id = Column(String, nullable=True)

#phase2B
retry_count = Column(Integer, nullable=False, default=0)
max_retries = Column(Integer, nullable=False, default=3)
timeout_seconds = Column(Integer, nullable=False, default=300)
last_error = Column(Text, nullable=True)

created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
updated_at = Column(
    DateTime,
    nullable=False,
    default=datetime.utcnow,
    onupdate=datetime.utcnow
)
execution_mode = Column(String, default="immediate") # immediate /
scheduled
scheduled_time = Column(DateTime, nullable=True)

steps = relationship("JobStep", back_populates="job", cascade="all,
delete-orphan")

class JobStep(Base):
    __tablename__ = "job_steps"

    id = Column(Integer, primary_key=True, autoincrement=True)
    job_id = Column(String, ForeignKey("jobs.id", ondelete="CASCADE"),
nullable=False, index=True)

    step_name = Column(String, nullable=False)
    status = Column(String, nullable=False, default="PENDING")
    message = Column(Text, nullable=True)
    exit_code = Column(Integer, nullable=True)

    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)

```

```

job = relationship("Job", back_populates="steps")

class JobStatus(str, enum.Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCESS = "SUCCESS"
    FAILED = "FAILED"
    VALIDATING = "VALIDATING"
    VERIFYING = "VERIFYING"
    FAILED_FINAL = "FAILED_FINAL"

```

9. executors

9.1. executors\linux_executors.py

```

import time
import paramiko

from app.executors.result_models import ExecutionResult
from app.services.credential_provider import get_linux_credentials

def run_linux(job):
    creds = get_linux_credentials()

    host = job.target_host
    username = creds["username"]
    password = creds["password"]
    private_key_path = creds["private_key_path"]

    start = time.time()

    client = paramiko.SSHClient()
    client.load_system_host_keys()
    # For production, prefer known_hosts / explicit host keys.
    # Do NOT switch to AutoAddPolicy unless explicitly accepted for controlled
environments.

    connect_kwargs = {
        "hostname": host,
        "username": username,
        "timeout": job.timeout_seconds or 300,
    }

    if private_key_path:
        key = paramiko.RSAKey.from_private_key_file(private_key_path)

```

```

        connect_kwargs["pkey"] = key
    else:
        connect_kwargs["password"] = password

    client.connect(**connect_kwargs)

    command = f"echo Installing {job.software_name} {job.software_version} or
''}"
    stdin, stdout, stderr = client.exec_command(command)

    exit_code = stdout.channel.recv_exit_status()
    out = stdout.read().decode(errors="ignore")
    err = stderr.read().decode(errors="ignore")

    client.close()

    duration = round(time.time() - start, 2)

    return ExecutionResult(
        success=(exit_code == 0),
        exit_code=exit_code,
        stdout=out,
        stderr=err,
        duration_seconds=duration,
        transport="ssh",
    )

```

9.2 executors\result_models.py

```

from pydantic import BaseModel
from typing import Optional

class ExecutionResult(BaseModel):
    success: bool
    exit_code: int
    stdout: Optional[str] = None
    stderr: Optional[str] = None
    duration_seconds: Optional[float] = None
    transport: Optional[str] = None

```

9.3 executors\windows_executor.py

```

import time
import winrm

from app.executors.result_models import ExecutionResult
from app.services.credential_provider import get_windows_credentials

def run_windows(job):
    creds = get_windows_credentials()

    host = job.target_host
    username = creds["username"]
    password = creds["password"]
    transport = creds["transport"]

    start = time.time()

    # Example package action – replace with controlled command builder
    ps_script = f"""
Write-Output "Installing {job.software_name} {job.software_version or ''}"
"""

    session = winrm.Session(
        target=host,
        auth=(username, password),
        transport=transport,
        server_cert_validation="ignore" # tighten for production certificate
trust
    )

    response = session.run_ps(ps_script)
    duration = round(time.time() - start, 2)

    return ExecutionResult(
        success=(response.status_code == 0),
        exit_code=response.status_code,
        stdout=response.std_out.decode(errors="ignore") if response.std_out
else "",
        stderr=response.std_err.decode(errors="ignore") if response.std_err
else "",
        duration_seconds=duration,
        transport="winrm",
    )

```

10. Queue

10.1 Job_queue.py

```

import logging
import queue
import threading
from typing import Callable

logger = logging.getLogger("app.queue")

_job_queue = queue.Queue()
_workers = []
_started = False

# NEW: track active jobs
_active_jobs = set()
_active_lock = threading.Lock()

def start_workers(worker_count: int, worker_handler: Callable[[str], None]):
    global _started
    if _started:
        return

    def worker():
        while True:
            job_id = _job_queue.get()

            if job_id is None:
                break

            thread_name = threading.current_thread().name

            print(f"💧 Worker picked job: {job_id}")
            print(f"🧵 Thread [{thread_name}] handling job: {job_id}")

            # mark active
            with _active_lock:
                _active_jobs.add(job_id)

            try:
                worker_handler(job_id)
            except Exception as exc:
                logger.exception("Worker failed for job_id=%s error=%s",
job_id, exc)
            finally:
                # remove from active
                with _active_lock:

```

```

        _active_jobs.discard(job_id)

        _job_queue.task_done()

for _ in range(worker_count):
    t = threading.Thread(target=worker, daemon=True)
    t.start()
    _workers.append(t)

_started = True
logger.info("Started %s workers", worker_count)

def enqueue_job(job_id: str):
    _job_queue.put(job_id)

    print(f"📌 Job queued: {job_id}")
    print(f"📊 Queue size: {_job_queue.qsize()}")

def get_queue_status():
    with _active_lock:
        return {
            "queue_size": _job_queue.qsize(),           # waiting
            "worker_count": len(_workers),              # total workers
            "active_jobs_count": len(_active_jobs),     # running now
            "active_job_ids": list(_active_jobs),       # optional visibility
        }

def stop_workers():
    for _ in _workers:
        _job_queue.put(None)

```

11. Routes

11.1 routes\jobs.py

```

import logging
import uuid
from fastapi import APIRouter, Depends, HTTPException, Request, status
from sqlalchemy.orm import Session
from app.db import get_db
from app.models import Job, JobStep, JobStatus

```

```

from app.schemas import JobCreate,
JobResponse, JobListResponse, JobStatusUpdate, ExecuteResponse, JobProgressResponse
from datetime import datetime
from app.services.job_lifecycle import update_job_status
from app.services.execution_service import execute_job
from app.queue.job_queue import enqueue_job

```

```

logger = logging.getLogger("app.routes.jobs")
router = APIRouter(prefix="/api/v1/jobs", tags=["jobs"])

```

```

@router.post("", response_model=JobResponse,
status_code=status.HTTP_201_CREATED)
def create_job(payload: JobCreate, request: Request, db: Session =
Depends(get_db)):
    trace_id = getattr(request.state, "trace_id", str(uuid.uuid4()))
    job = Job(
        id=str(uuid.uuid4()),
        ticket_id=payload.ticket_id,
        module="sw-install",
        status=JobStatus.PENDING,
        target_host=payload.target_host,
        os_type=payload.os_type.lower(),
        software_name=payload.software_name,
        software_version=payload.software_version,
        requested_by=payload.requested_by,
        justification=payload.justification,
        #trace_id=getattr(request.state, "trace_id", str(uuid.uuid4())),
        trace_id=trace_id,
        #below optional
        created_at=datetime.utcnow(),
        updated_at=datetime.utcnow(),
        retry_count=0,
        max_retries=payload.max_retries or 3,
        timeout_seconds=payload.timeout_seconds or 300,
        execution_mode=payload.execution_mode or "immediate",
        scheduled_time=payload.scheduled_time,
    )

    db.add(job)
    db.flush()

    initial_step = JobStep(
        job_id=job.id,
        step_name="job_received",
        status="SUCCESS",

```

```

        message="Job request accepted and stored",
        exit_code=0,
        #below optional
        created_at=datetime.utcnow(),
    )
    db.add(initial_step)

    db.commit()
    db.refresh(job)
    logger.info("job_created job_id=%s ticket_id=%s trace_id=%s", job.id,
job.ticket_id, trace_id)
    return job

@router.get("", response_model=JobListResponse)
def list_jobs(db: Session = Depends(get_db)):
    jobs = db.query(Job).order_by(Job.created_at.desc()).all()
    return JobListResponse(items=jobs, total=len(jobs))

@router.get("/{job_id}", response_model=JobResponse)
def get_job(job_id: str, db: Session = Depends(get_db)):
    job = db.query(Job).filter(Job.id == job_id).first()
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")
    return job

@router.patch("/{job_id}/status", response_model=JobResponse)
def patch_job_status(job_id: str, payload: JobStatusUpdate, db: Session =
Depends(get_db)):
    job = db.query(Job).filter(Job.id == job_id).first()
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")
    try:
        job = update_job_status(db=db, job=job, new_status=payload.status,
message=payload.message)
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc))

    logger.info("job_status_updated job_id=%s new_status=%s", job.id,
job.status.value)
    return job

@router.post("/{job_id}/simulate-run", response_model=JobResponse)
def simulate_job_run(job_id: str, db: Session = Depends(get_db)):
    job = db.query(Job).filter(Job.id == job_id).first()
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")

    try:

```

```

        if job.status == JobStatus.PENDING:
            job = update_job_status(db=db, job=job,
new_status=JobStatus.VALIDATING, message="Validation started")
            if job.status == JobStatus.VALIDATING:
                job = update_job_status(db=db, job=job,
new_status=JobStatus.RUNNING, message="Execution started")
                if job.status == JobStatus.RUNNING:
                    job = update_job_status(db=db, job=job,
new_status=JobStatus.VERIFYING, message="Verification started")
                    if job.status == JobStatus.VERIFYING:
                        job = update_job_status(db=db, job=job,
new_status=JobStatus.SUCCESS, message="Job completed successfully")
            except ValueError as exc:
                raise HTTPException(status_code=400, detail=str(exc))

        return job

@router.post("/{job_id}/execute", response_model=ExecuteResponse)
def execute(job_id: str, db: Session = Depends(get_db)):
    job = db.query(Job).filter(Job.id == job_id).first()
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")

    if job.execution_mode == "scheduled" and job.scheduled_time:
        if job.scheduled_time > datetime.utcnow():
            raise HTTPException(status_code=400, detail="Job is scheduled for
later execution")
        if job.status in [JobStatus.SUCCESS, JobStatus.FAILED,
JobStatus.FAILED_FINAL]:
            raise HTTPException(status_code=400, detail="Job already completed")

    enqueue_job(job.id)
    return ExecuteResponse(job_id=job.id, message="Job queued for execution")

@router.get("/{job_id}/progress", response_model=JobProgressResponse)
def get_job_progress(job_id: str, db: Session = Depends(get_db)):
    job = db.query(Job).filter(Job.id == job_id).first()

    if not job:
        raise HTTPException(status_code=404, detail="Job not found")

    steps = db.query(JobStep).filter(JobStep.job_id ==
job_id).order_by(JobStep.created_at).all()

    return {
        "job_id": job.id,
        "status": job.status,

```

```

        "trace_id": job.trace_id,
        "last_error": job.last_error,
        "retry_count": job.retry_count,
        "max_retries": job.max_retries,

        "steps": [
            {
                "id": s.id,
                "step_name": s.step_name,
                "status": s.status,
                "message": s.message,
                "exit_code": s.exit_code,
                "created_at": s.created_at,
            }
            for s in steps
        ]
    }
}

```

11.2 routs\monitor_api.py

```

from fastapi import APIRouter
from app.queue.job_queue import get_queue_status

from app.models import Job
from app.db import SessionLocal

router1= APIRouter(prefix="/api/v1/monitor", tags=["monitoring"])

@router1.get("/queue")
def get_queue():
    return get_queue_status()

@router1.get("/active-jobs")
def get_active_jobs():
    db = SessionLocal()

    jobs = db.query(Job).filter(
        Job.status.in_(["VALIDATING", "RUNNING", "VERIFYING"])
    ).all()

    return [
        {
            "job_id": j.id,
            "status": j.status

```

```

    }
    for j in jobs
]

```

12. Services

12.1 services\credential_provider.py

```

import os

def get_windows_credentials():
    return {
        "username": os.getenv("WINRM_USERNAME"),
        "password": os.getenv("WINRM_PASSWORD"),
        "transport": os.getenv("WINRM_TRANSPORT", "ntlm"),
    }

def get_linux_credentials():
    return {
        "username": os.getenv("SSH_USERNAME"),
        "password": os.getenv("SSH_PASSWORD"),
        "private_key_path": os.getenv("SSH_PRIVATE_KEY_PATH"),
    }

```

12.2 services\execution_service.py

```

import logging
import time
from datetime import datetime
from app.db import SessionLocal
from app.models import Job, JobStatus
from app.executors import windows_executor, linux_executor
from app.services.job_lifecycle import update_job_status, append_job_step

logger = logging.getLogger("app.services.execution")

def execute_job(job):
    os_type = (job.os_type or "").strip().lower()

    print(f"🤔 Deciding executor for OS: {os_type}")

    if os_type == "windows":
        return windows_executor.run_windows(job)
    elif os_type == "linux":
        return linux_executor.run_linux(job)

```

```

else:
    raise ValueError(f"Unsupported os_type: {job.os_type}")

def mark_job_failed(db, job, error_message: str):
    """
    Persist failure details so progress API shows FAILED/FAILED_FINAL
    instead of leaving the job stuck in RUNNING.
    """
    # increment retry count
    job.retry_count = (job.retry_count or 0) + 1
    job.last_error = error_message

    if hasattr(job, "completed_at"):
        job.completed_at = datetime.utcnow()

    db.add(job)
    db.commit()
    db.refresh(job)

    # append explicit failed execution step
    append_job_step(
        db=db,
        job_id=job.id,
        step_name="execution_failed",
        status="FAILED",
        message=error_message,
        exit_code=1,
    )
    db.commit()

    # choose terminal status based on retry count
    if job.max_retries and job.retry_count >= job.max_retries:
        final_status = JobStatus.FAILED_FINAL
        final_message = f"Execution failed after {job.retry_count} attempt(s): {error_message}"
    else:
        final_status = JobStatus.FAILED
        final_message = error_message

    if job.status != final_status:
        job = update_job_status(db, job, final_status, final_message)

    return job

def process_job(job_id: str):
    print(f"☑ process_job started for: {job_id}")
    db = SessionLocal()

```

```

try:
    job = db.query(Job).filter(Job.id == job_id).first()
    if not job:
        print("✖ Job not found")
        return

    print(f"🔗 Current status: {job.status}")

    # Step 1 – VALIDATING
    if job.status == JobStatus.PENDING:
        print("➡ Moving to VALIDATING")
        job = update_job_status(db, job, JobStatus.VALIDATING, "Validation
started")

        db.refresh(job)
        time.sleep(10)

    # Step 2 – RUNNING
    if job.status == JobStatus.VALIDATING:
        print("➡ Moving to RUNNING")
        job = update_job_status(db, job, JobStatus.RUNNING, "Execution
started")

        db.refresh(job)
        time.sleep(15)

    # Step 3 – EXECUTION
    print("⚡ Executing...")
    start_time = time.time()

    result = execute_job(job)

    execution_time = round(time.time() - start_time, 2)
    print(f"✅ Execution result: {result}")
    print(f"🕒 Execution took {execution_time} seconds")

    append_job_step(
        db=db,
        job_id=job.id,
        step_name="execution_time",
        status="SUCCESS",
        message=f"Execution took {execution_time} seconds",
        exit_code=0
    )
    db.commit()

    # Treat any non-success executor return as a failure
    if str(result).upper() != "SUCCESS":
        raise RuntimeError(f"Executor returned non-success result:
{result}")

```

```

time.sleep(10)

# Step 4 – VERIFYING
if job.status == JobStatus.RUNNING:
    print("➡ Moving to VERIFYING")
    job = update_job_status(db, job, JobStatus.VERIFYING,
"Verification started")
    db.refresh(job)
    time.sleep(10)

# Step 5 – FINAL SUCCESS
if job.status == JobStatus.VERIFYING:
    print("✅ Moving to SUCCESS")
    job.last_error = None
    if hasattr(job, "completed_at"):
        job.completed_at = datetime.utcnow()
    db.add(job)
    db.commit()
    db.refresh(job)

    job = update_job_status(db, job, JobStatus.SUCCESS, "Execution
completed")
    post_execution_handler(job, "SUCCESS")

except Exception as e:
    error_message = str(e)
    print(f"💥 ERROR: {error_message}")
    logger.exception("Job execution failed", extra={"job_id": job_id})

    if "job" in locals() and job:
        mark_job_failed(db, job, error_message)

finally:
    db.close()

def post_execution_handler(job, final_status: str):
    """
    Placeholder for downstream enterprise workflow integration.
    Future use:
    - close SCTask
    - create CR
    - trigger ops validation workflow
    - trigger manual fallback ticketing
    """
    print(f"📦 post_execution_handler invoked for job={job.id},
final_status={final_status}")

```

12.3 services\job_lifecycle.py

```

from sqlalchemy.orm import Session
from app.models import Job, JobStatus, JobStep

ALLOWED_TRANSITIONS = {
    JobStatus.PENDING: [JobStatus.VALIDATING, JobStatus.FAILED,
JobStatus.FAILED_FINAL],
    JobStatus.VALIDATING: [JobStatus.RUNNING, JobStatus.FAILED,
JobStatus.FAILED_FINAL],
    JobStatus.RUNNING: [JobStatus.VERIFYING, JobStatus.FAILED,
JobStatus.FAILED_FINAL],
    JobStatus.VERIFYING: [JobStatus.SUCCESS, JobStatus.FAILED,
JobStatus.FAILED_FINAL],
    JobStatus.SUCCESS: [],
    JobStatus.FAILED: [JobStatus.FAILED_FINAL],
    JobStatus.FAILED_FINAL: [],
}

def validate_status_transition(current_status: JobStatus, new_status:
JobStatus):
    allowed = ALLOWED_TRANSITIONS.get(current_status, [])
    if new_status not in allowed:
        raise ValueError(
            f"Invalid status transition: {current_status.value} ->
{new_status.value}"
        )

def append_job_step(db: Session, job_id: str, step_name: str, status: str,
message: str | None = None, exit_code: int | None = 0):
    step = JobStep(
        job_id=job_id,
        step_name=step_name,
        status=status,
        message=message,
        exit_code=exit_code,
    )
    db.add(step)
    return step

def update_job_status(db: Session, job: Job, new_status: JobStatus, message:
str | None = None):
    validate_status_transition(job.status, new_status)

```

```

job.status = new_status
db.add(job)

append_job_step(
    db=db,
    job_id=job.id,
    step_name=f"status_{new_status.value.lower()}",
    status=new_status.value,
    message=message,
    exit_code=0,
)

db.commit()
db.refresh(job)
return job

```

13. env (development testing)

```

APP_NAME=software-install-mvp
APP_ENV=local
APP_HOST=0.0.0.0
APP_PORT=8000
LOG_LEVEL=INFO
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=software_install
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
WINRM_USERNAME=Administrator
WINRM_PASSWORD=your_password_here
WINRM_TRANSPORT=ntlm
SSH_USERNAME=your_ssh_username
SSH_PASSWORD=your_ssh_password
SSH_PRIVATE_KEY_PATH=/path/to/your/private/key

```

14.requirements.txt

```

fastapi==0.115.0
uvicorn[standard]==0.30.6
sqlalchemy==2.0.35
psycpg2-binary==2.9.12
pydantic==2.9.2
pydantic-settings==2.5.2
python-dotenv==1.0.1
pip-system-certs
alembic==1.13.2
pywinrm
paramiko

```

15.docker-compose.yml

```
version: "3.9"
services:
  postgres:
    image: postgres:16
    container_name: software-install-postgres
    environment:
      POSTGRES_DB: software_install
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

16.Testing:

```
docker login
docker compose up -d
activate venv
uvicorn app.main:app -reload
test http://127.0.0.1:8000/ //browser and check terminal its should be ok
```

```
enqueue_job(job_id)
↓
queue receives job ☑
↓
worker picks job instantly ☑
↓
queue becomes empty
```

a.create a fres job

```
-----
$response = Invoke-RestMethod `
-Uri "http://127.0.0.1:8000/api/v1/jobs" `
-Method Post `
-ContentType "application/json" `
-Body (@{
    ticket_id = "RITM23215"
    target_host = "server01.company.net"
    os_type = "windows"
    software_name = "7zip"
} | ConvertTo-Json)
```

`$response`

b.

2. Queue the job for execution

Invoke-RestMethod `

-Uri "http://127.0.0.1:8000/api/v1/jobs/\${\$response.id}/execute" `

-Method Post

c.

3. Poll job status

Invoke-RestMethod `

-Uri "http://127.0.0.1:8000/api/v1/jobs/\${\$response.id}" `

-Method Get

d.

4. List jobs

Invoke-RestMethod `

-Uri "http://127.0.0.1:8000/api/v1/jobs" `

-Method Get

use this to view the complete report

Invoke-RestMethod `

-Uri "http://127.0.0.1:8000/api/v1/jobs" `

-Method Get | ConvertTo-Json -Depth 5

test api monitoring

Invoke-RestMethod `

-Uri "http://127.0.0.1:8000/api/v1/monitor/queue" `

-Method Get | ConvertTo-Json

Invoke-RestMethod `

-Uri "http://127.0.0.1:8000/api/v1/jobs/\${\$response.id}/progress" `

-Method Get | ConvertTo-Json -Depth 5

In real systems, queue is NOT visible directly – it's inferred via behavior

```
alembic init alembic //it will create folder and update .env file
maintain versions, while making changes on models, below command will
automaticlaly create vesions
alembic revision --autogenerate -m "add execution fields" //rename revision
filename ass uniform and updt revision value in same file
```

```
alembic upgrade head( update model, then above and then this command)
```

[17. Master context docs/Master_Context_Phase2_Updated.docx](#)

Updated Master Context & Phase 2 Plan – Agentic AI Software Installation Platform

1. Project Overview

The platform is designed to automate software installation across Windows and Linux systems using a job-driven architecture. It integrates API, lifecycle management, execution engines, and future AI-based automation.

2. Current Implementation Status (Completed Phases)

Phase 1 & Phase 1B are fully completed:

- FastAPI API layer implemented
- PostgreSQL database setup with jobs and job_steps tables
- Job lifecycle management (PENDING → VALIDATING → RUNNING → VERIFYING → SUCCESS/FAILED)
- Worker queue system for asynchronous processing
- Retry logic and failure handling
- Structured logging and trace_id tracking
- Progress API with detailed step visibility
- Alembic migrations enabled

3. Current Architecture Flow

Client / ServiceNow → FastAPI → Job Lifecycle Service → Queue → Worker → Execution Service → Executors → Target Systems

PostgreSQL acts as the system of record.

4. Phase 2 Objective

Introduce real execution capability to install software remotely on Windows and Linux systems while maintaining existing lifecycle and observability architecture.

5. Phase 2 Features (Detailed)

5.1 Executor Framework

- Base executor interface
- Factory pattern for OS selection
- Separation of execution logic from services

5.2 Windows Execution

- WinRM integration
- NTLM authentication
- PowerShell command execution

- Credentials managed via .env (temporary)

5.3 Linux Execution

- SSH / Paramiko-based connectivity
- Remote command execution

5.4 Execution Flow

Worker → Execution Service → Executor Factory → OS Executor → Remote System Execution

5.5 Error Handling

- Capture execution errors
- Persist in job.last_error
- Append failure steps (execution_failed)
- Transition job to FAILED / FAILED_FINAL

5.6 Retry Mechanism

- Use retry_count and max_retries
- Transition to FAILED_FINAL after max attempts

5.7 Timeout Handling

- Use timeout_seconds to control execution duration

5.8 Observability

- Step-level execution tracking
- Execution timing capture
- Trace ID correlation across logs

6. Target Code Structure (Phase 2)

/app

/executors

base_executor.py

windows_executor.py

linux_executor.py

executor_factory.py

/services

execution_service.py

job_lifecycle.py

/queue

/routes

/schemas

/models

7. Design Principles

- API-first design
- Separation of concerns
- No execution logic in API routes
- Modular extensibility
- Lifecycle-driven execution
- Observability-first approach

8. Testing Strategy for Phase 2

- Validate Windows execution success path
- Validate Linux execution success path
- Simulate failures and verify error capture
- Validate retry logic behavior
- Validate timeout scenarios

9. Future Roadmap

Phase 3 – Credential Security

- Azure Key Vault integration
- Remove credentials from code

Phase 4 – SOP + AI Integration

- SOP file parsing
- Azure OpenAI integration

Phase 5 – MCP Architecture

- Convert executors into MCP tools
- AI agent-driven execution

10. Conclusion

The system has a strong and stable foundation. Phase 2 focuses on introducing real execution while maintaining scalability, modularity, and observability.